

The purpose of this assignment is to build a calculator for C integer expressions using yacc and lex. The program reads a series of statements and prints the result of each one. It supports 26 predefined integer variables (a through z), the full set of C integer operators (math operations, bitwise operations, etc.), and two extra commands: dump and clear. On top of parsing the expressions correctly, the calculator also has to detect integer overflow and division by zero, report the first error encountered instead of a result, and skip any assignments that would happen after that error.

Precedence and associativity was designed by how the grammar itself is structured. A ladder of nonterminals was built, one per row of the operator table, from lowest precedence down to highest: or_expr, xor_expr, and_expr, shift_expr, add_expr, mul_expr, neg_expr, not_expr, and primary for parentheses, numbers, and variables. Each level only knows how to combine itself with the level below it, which is what gives left associativity, since the recursive call is on the left.

Assignment doesn't fit into that ladder, since it's right-associative and has to bind more loosely than everything else. Instead of adding it as another rung, I placed it above the whole ladder as its own set of productions on expr: VAR '=' expr, VAR '+=' expr, and so on, with the right-hand side recursing back into expr and the left-hand side always a bare variable. Since the only way into an assignment production is through expr, and the only way out of it is into another assignment or an or_expr, there's no path in the grammar that lets an operator appear before an assignment. That restriction the assignment calls for falls out of the grammar shape rather than needing an explicit check. I had issues wrapping my head around this portion both conceptually and applying it, so Claude was used to help implement this piece.

Error handling uses a single global flag that gets reset before each calculation is evaluated. Since yacc builds results bottom-up, there isn't another natural place to run code before the expression starts being evaluated. Every mathematic and shift helper function checks that flag first and exits immediately if it's already set, so once an error happens nothing after it does any real work. Overflow is caught by computing every add, subtract, multiply, and shift in a wider 64-bit type and checking whether the result still fits in an int. Divide and modulo separately check for a zero divisor. Assignments only commit to a variable if no error occurred while computing the value, which is what keeps a failed calculation from partially updating a variable.

The provided input.txt and its expected output were the first check, and matched exactly. From there I used Claude to write a series of test files that isolate one thing at a time: every operator on its own, operator precedence and left/right associativity together (parenthesized vs. unparenthesized versions of the same expression), every compound assignment operator, chained assignment, and the dump and clear commands. I also targeted overflow, literals just past INT_MAX, addition/subtraction/multiplication that cross the boundary, large shifts, and negating INT_MIN, plus division and modulo by zero. I ran each test file through both my program to verify all tests passed before considering the assignment as complete.